Guía de uso · versión `base`

Paso a paso para usar `franki678/bigdata-sandbox:base` (en Docker Hub, `latest` es la misma imagen): instalar Docker, tener la imagen, crear el cluster y recorrer cada herramienta con los ejemplos que trae la imagen. Leela en orden: cada parte usa lo que dejó la anterior. Al final están los problemas comunes y cómo empezar de cero.

README de `base` · Mapa interactivo · Glosario · README del proyecto

Los comandos de esta guía los corre el CI en cada cambio, en este orden, contra la imagen publicada (`pruebas/tutoriales.py`). Los números que aparecen como resultado (200000 filas, 448780000 de facturación de la yerba...) salen de esas corridas: si te da otra cosa, algo cambió.

Contenido

1. Antes de empezar
2. Instalar Docker
3. Tener la imagen
4. Crear el cluster
5. Entrar, mirar y las páginas web
6. Apagar, prender y cambiar la arquitectura
7. HDFS
8. YARN y MapReduce
9. Pig
10. Hive
11. MySQL y Sqoop
12. Spark
13. MongoDB
14. ZooKeeper y alta disponibilidad
15. Problemas comunes
16. Empezar de cero

---

1. Antes de empezar

Qué necesitás

Qué	Cuánto
Sistema	Windows 10/11 de 64 bits, Linux, o Mac (Intel o M1 a M4)
RAM para Docker	**7 GB** con MongoDB (como en esta guía), 6 GB sin MongoDB
Disco libre	~4 GB en Windows y Mac, ~2,5 GB en Linux

Qué	Cuánto
Procesador	Para MongoDB, Intel/AMD con **AVX** (casi todos desde 2011) o un ARM moderno. Sin AVX anda todo menos MongoDB

Los números de RAM y disco salen del workflow `Imagen` del repo, que los mide en cada versión nueva (el detalle está en el README de `base`).

Dos terminales: tu compu y adentro

Todo lo que empieza con `docker` va en la terminal de **tu compu** (PowerShell o CMD en Windows, la Terminal en Mac y Linux). El resto (`hdfs`, `pig`, `beeline`, `mysql`, `mongosh`, `estado`…) va **adentro** del contenedor, después de `docker exec -it bigdata bash`. Cada bloque de esta guía dice dónde va.

Cuándo	Qué (en tu compu)
Una sola vez	Instalar Docker, tener la imagen y el `docker run` de la parte 4
Al empezar cada clase	`docker start bigdata` y esperar el cartel **Cluster LISTO**
Para entrar (las veces que quieras)	`docker exec -it bigdata bash`
Al terminar cada clase	`exit` y `docker stop bigdata`

---

2. Instalar Docker

Windows

1. Bajá **Docker Desktop** de https://www.docker.com/products/docker-desktop/ e instalalo con las opciones que trae (usa WSL 2). Pide reiniciar.
2. Abrilo y esperá a que abajo a la izquierda diga *Engine running*.
3. Si dice que falta WSL o que la virtualización está apagada, en una consola **como administrador**:

   ```
   wsl --install
   ```

   y reiniciá. Si sigue, hay que prender la virtualización (*Intel VT-x* o *AMD-V/SVM*) en el BIOS.

La memoria en Windows la decide WSL, no Docker Desktop (por defecto, la mitad de la RAM de la compu). Con 16 GB ya alcanza. Con 8 GB, o para fijarla, creá el archivo `%UserProfile%\.wslconfig` (sin extensión) con esto, cerrá Docker Desktop, corré `wsl --shutdown` y volvé a abrirlo:

```
[wsl2]
memory=7GB
```

Mac

Docker Desktop del mismo link (*Apple Silicon* o *Intel*, según tu Mac). En *Settings → Resources → Memory*, 7 GB o más.

Linux

Docker Engine, con el paquete de tu distribución o con https://docs.docker.com/engine/install/. No tiene tope de memoria: usa la de la compu. Para no escribir `sudo` en cada comando: `sudo usermod -aG docker $USER` y volver a entrar.

Comprobar que anda

```
docker version
```

Tienen que aparecer dos partes, *Client* y *Server*. Si sólo aparece *Client* y un error, Docker Desktop no está abierto.

¿Tu procesador tiene AVX? En Linux, si esto imprime `avx`, sí:

```
grep -o -m1 -w avx /proc/cpuinfo
```

En Windows no hace falta mirarlo: si no tiene, al arrancar con MongoDB el log dice `AVISO: este procesador no tiene AVX` y sigue sin MongoDB. Las Mac M1 a M4 siempre andan.

---

3. Tener la imagen

Tres formas; las tres terminan con una imagen llamada `franki678/bigdata-sandbox:base` en tu compu. Escribila siempre **con el** `:base`: sin tag, Docker busca `:latest` y sale a internet.

A. De Docker Hub (con internet)

```
docker pull franki678/bigdata-sandbox:base
```

Baja ~1,4 GB, sin cuenta ni `docker login`. Docker elige solo la variante de tu procesador (Intel/AMD o ARM).

B. En el aula: el archivo de la imagen

Si 20 PCs bajan la imagen a la vez se satura la red y Docker Hub corta las descargas sin cuenta. Por eso la imagen `base` también está en **un solo archivo**, para Intel/AMD (Windows o Linux), en el release `imagen` del repo:

https://github.com/Franco-SanMartin/bigdata-sandbox/releases/download/imagen/bigdata-sandbox.tar.gz

Lo arma el workflow `Aula` (`.github/workflows/aula.yml`) con la `base` ya publicada: la guarda con `docker save` con un solo nombre (`franki678/bigdata-sandbox:base`), prueba que se baje y se cargue bien, y la sube reemplazando la anterior.

1. **La PC del profesor lo baja** con el navegador (~1,4 GB). Si esa PC se borra al reiniciar, conviene tenerlo también en un pendrive.
2. **Lo reparte**, de alguna de estas formas:
    - **Veyon:** seleccionar todas las PCs → *Transferencia de archivos* → `bigdata-sandbox.tar.gz` → *Solo transferir*. Queda en la carpeta del usuario de cada PC.
    - **Pendrive:** entra en uno de 4 GB, aunque sea FAT32.
    - **Por la red, desde la PC del profesor** (con la imagen ya cargada ahí). En la carpeta donde está el archivo, en CMD:

```
docker run -d --rm --name compartir -p 8000:8000 -v "%cd%:/srv:ro" -w /srv --
entrypoint python3 franki678/bigdata-sandbox:base -m http.server 8000
ipconfig
```

De `ipconfig`, la *Dirección IPv4* (por ejemplo `192.168.1.10`). Cada alumno lo baja con su IP:

```
curl.exe -o bigdata-sandbox.tar.gz http://192.168.1.10:8000/bigdata-
sandbox.tar.gz
```

Al terminar, el profesor corta el servidor con `docker stop compartir`.

3. **Cada alumno lo carga**, con Docker Desktop abierto, en la carpeta donde quedó el archivo:

   ```
   docker load -i bigdata-sandbox.tar.gz
   ```

   Tarda 1 o 2 minutos y termina con `Loaded image: franki678/bigdata-sandbox:base`. Después el archivo se puede borrar.

Para Mac M1 a M4 (ARM) no hay archivo: con internet, la forma A.

C. Entre compañeros

```
docker save franki678/bigdata-sandbox:base -o bigdata-sandbox.tar
docker load -i bigdata-sandbox.tar
```

Comprobar que está

```
docker image ls franki678/bigdata-sandbox
```

Tiene que aparecer una fila con `franki678/bigdata-sandbox` y `base`. Si cargaste un archivo viejo que traía también `latest` con el mismo IMAGE ID, `docker rmi franki678/bigdata-sandbox:latest` saca sólo ese nombre.

---

4. Crear el cluster

La carpeta de trabajo

Creá una carpeta y abrí una terminal parada ahí (en Windows, en el Explorador: clic en la barra de dirección, escribir `cmd` y Enter). Adentro de ella, `datos/` va a ser la carpeta compartida con el cluster.

```
mkdir bigdata
cd bigdata
```

El `docker run`

Un cluster con 3 DataNodes, cada bloque guardado 2 veces, bloques de 1 MB (para ver muchos bloques con archivos chicos) y MongoDB prendido. Elegí el de tu consola.

Mac y Linux:

```
docker run -d --name bigdata --stop-timeout 20 \
  -e DATANODES=3 -e REPLICATION=2 -e BLOCK_SIZE=1m -e MONGODB=si \
  -p 127.0.0.1:9870:9870 -p 127.0.0.1:9868:9868 -p 127.0.0.1:9901-9909:9901-9909 \
  -p 127.0.0.1:8088:8088 -p 127.0.0.1:8042:8042 -p 127.0.0.1:19888:19888 \
  -p 127.0.0.1:10000:10000 -p 127.0.0.1:10002:10002 \
  -p 127.0.0.1:18080:18080 -p 127.0.0.1:4040:4040 \
  -p 127.0.0.1:3306:3306 -p 127.0.0.1:27017:27017 \
  -v bigdata_hadoop:/hadoop -v "$(pwd)/datos:/datos" \
  franki678/bigdata-sandbox:base
```

Windows, PowerShell (una línea):

```
docker run -d --name bigdata --stop-timeout 20 -e DATANODES=3 -e REPLICATION=2 -e
BLOCK_SIZE=1m -e MONGODB=si -p 127.0.0.1:9870:9870 -p 127.0.0.1:9868:9868 -p 127.0.0.1:9901-
9909:9901-9909 -p 127.0.0.1:8088:8088 -p 127.0.0.1:8042:8042 -p 127.0.0.1:19888:19888 -p
127.0.0.1:10000:10000 -p 127.0.0.1:10002:10002 -p 127.0.0.1:18080:18080 -p
127.0.0.1:4040:4040 -p 127.0.0.1:3306:3306 -p 127.0.0.1:27017:27017 -v
bigdata_hadoop:/hadoop -v "${PWD}/datos:/datos" franki678/bigdata-sandbox:base
```

Windows, CMD (una línea):

```
docker run -d --name bigdata --stop-timeout 20 -e DATANODES=3 -e REPLICATION=2 -e
BLOCK_SIZE=1m -e MONGODB=si -p 127.0.0.1:9870:9870 -p 127.0.0.1:9868:9868 -p 127.0.0.1:9901-
9909:9901-9909 -p 127.0.0.1:8088:8088 -p 127.0.0.1:8042:8042 -p 127.0.0.1:19888:19888 -p
127.0.0.1:10000:10000 -p 127.0.0.1:10002:10002 -p 127.0.0.1:18080:18080 -p
127.0.0.1:4040:4040 -p 127.0.0.1:3306:3306 -p 127.0.0.1:27017:27017 -v
bigdata_hadoop:/hadoop -v "%cd%\datos:/datos" franki678/bigdata-sandbox:base
```

Lo único que cambia es cómo se escribe "la carpeta actual": `$(pwd)`, `${PWD}` o `%cd%`. Con el de otra consola, Docker dice `invalid characters for a local volume name`.

Parte	Qué hace
`-d`	Corre en segundo plano y te devuelve la consola
`--name bigdata`	El nombre del contenedor: con ese se prende, se apaga y se entra
`--stop-timeout 20`	20 s para apagarse en orden (MySQL, MongoDB, Hive, YARN, HDFS) antes de que Docker corte
`-e ...`	La arquitectura del cluster (tabla de abajo)
`-p 127.0.0.1:9870:9870` …	Cada página web (y MySQL y MongoDB) en tu compu. El `127.0.0.1` hace que se vean **sólo desde tu compu**: el sandbox no tiene contraseñas
`-v bigdata_hadoop:/hadoop`	El **volumen**: el disco del cluster. HDFS, el metastore de Hive, MySQL y MongoDB viven ahí y sobreviven a apagar y a borrar el contenedor
`-v "$(pwd)/datos:/datos"`	Tu carpeta `datos/` es `/datos` adentro: por ahí se pasan archivos
`franki678/bigdata-sandbox:base`	La imagen

Los parámetros

Todos opcionales. Los lee el arranque (`entrypoint.sh` y `sandbox/lib.sh`); si uno está mal escrito, el contenedor se apaga y `docker logs bigdata` dice cuál (`[sandbox] ERROR: ...`).

Parámetro	Qué elige	Por defecto	Valores
`DATANODES`	Cuántos DataNodes (guardan los bloques; ~250 MB de RAM cada uno)	`1`	`1` a `9`
`REPLICATION`	Copias de cada bloque, cada una en un DataNode distinto	`1`	`1` a `9`
`BLOCK_SIZE`	Tamaño de bloque	`128m`	número + `m` o `g` , mínimo `1m`
`NAMENODES`	NameNodes. Con 2 o 3 hay alta disponibilidad (parte 14). Queda fijo al crear el volumen	`1`	`1` a `3`
`SERVICIOS`	Qué se prende además de HDFS: `hdfs` (sólo HDFS, 4 GB), `mapreduce` (+ YARN, JobHistory, MySQL, 5 GB), `todo` (+ Hive y Spark, 6 GB)	`todo`	`hdfs` , `mapreduce` , `todo`
`MONGODB`	MongoDB con la base de ejemplo `tienda` (+1 GB)	`no`	`si` , `no`
`PIG_HEAPSIZE`	Memoria en MB del comando `pig` (no de sus trabajos)	`512`	número

`ROL` , `RED` , `TRABAJADOR_RAM` y los demás del cluster de varias PCs son de la versión `empresa` ; `JUPYTER` , de la `ml` .

Esperar el cartel "Cluster LISTO"

La primera vez formatea HDFS y carga las bases de ejemplo; después sólo prende los procesos. Para mirarlo:

```
docker logs -f bigdata
```

Cuando aparece **Cluster LISTO**, abajo se ve cada proceso con su estado y su página. `Ctrl+C` sale del log; el cluster sigue prendido.

Con docker compose (opcional)

El `docker-compose.yml` del repo arma el mismo cluster, con cada parámetro y cada puerto comentado. Bajalo a tu carpeta de trabajo y:

```
docker compose up -d      # crear y prender
docker compose logs -f    # mirar el arranque
docker compose stop       # apagar
docker compose start      # prender
docker compose down       # borrar el contenedor (el volumen queda)
```

Usa el mismo nombre de contenedor (`bigdata`) y el mismo volumen (`bigdata_hadoop`) que el `docker run` : o uno o el otro. Para pasar de uno al otro, antes `docker rm -f bigdata` (los datos quedan).

5. Entrar, mirar y las páginas web

`estado` lista cada proceso, si está arriba y su página. Se puede correr desde tu compu sin entrar:

```
docker exec bigdata estado
```

Para entrar:

```
docker exec -it bigdata bash
```

El prompt cambia a `root@...:/datos#`: estás adentro, en `/datos` (tu carpeta `datos/`). Los ejemplos se copian ahí solos al arrancar, sin pisar los que ya tengas:

```
ls ejemplos
estado
jps
```

`jps` lista los procesos Java, uno por nodo: `NameNode (nn1)`, `DataNode (dn2)`... (es una versión propia del sandbox, que agrega la instancia). `exit` vuelve a tu compu; el cluster sigue prendido.

Con el cluster prendido, en el navegador de tu compu:

Componente	Dirección	Qué mirar
NameNode	http://localhost:9870	*Datanodes*: los nodos vivos. *Utilities → Browse the file system*: cada archivo y sus bloques
Secondary NameNode	http://localhost:9868	El último *checkpoint* de los metadatos
DataNode N	http://localhost:9901 a 9909	DataNode 1 → 9901, DataNode 2 → 9902...
YARN ResourceManager	http://localhost:8088	Todos los trabajos: MapReduce, Pig, Hive (Tez), Spark, Sqoop
NodeManager	http://localhost:8042	Los contenedores de YARN corriendo
JobHistory	http://localhost:19888	Trabajos MapReduce terminados y sus logs
HiveServer2	http://localhost:10002	Sesiones y consultas de Hive
Spark History	http://localhost:18080	Trabajos de Spark terminados
Spark activo	http://localhost:4040	Sólo mientras corre un trabajo de Spark
MySQL	`localhost:3306`	Para DBeaver o Workbench: usuario `alumno`, clave `alumno`, base `tienda`
MongoDB	`localhost:27017`	Para MongoDB Compass: `mongodb://localhost:27017`

6. Apagar, prender y cambiar la arquitectura

Al terminar:

```
docker stop bigdata
```

Apaga cada servicio en orden (hasta 20 s). Los datos quedan en el volumen. La próxima vez:

```
docker start bigdata
```

y esperar otra vez **Cluster LISTO**. No repitas el `docker run`: falla con `the container name "/bigdata" is already in use`. Al prender, HDFS pasa unos segundos en *safe mode* revisando los bloques; el arranque lo espera solo.

Cambiar los parámetros

Se borra el **contenedor** y se crea de nuevo con otros `-e`. El **volumen** queda: HDFS, las tablas de Hive y las bases siguen ahí. De 3 a 4 DataNodes:

```
docker rm -f bigdata
docker run -d --name bigdata --stop-timeout 20 \
  -e DATANODES=4 -e REPLICATION=2 -e BLOCK_SIZE=1m -e MONGODB=si \
  -p 127.0.0.1:9870:9870 -p 127.0.0.1:9868:9868 -p 127.0.0.1:9901-9909:9901-9909 \
  -p 127.0.0.1:8088:8088 -p 127.0.0.1:8042:8042 -p 127.0.0.1:19888:19888 \
  -p 127.0.0.1:10000:10000 -p 127.0.0.1:10002:10002 \
  -p 127.0.0.1:18080:18080 -p 127.0.0.1:4040:4040 \
  -p 127.0.0.1:3306:3306 -p 127.0.0.1:27017:27017 \
  -v bigdata_hadoop:/hadoop -v "$(pwd)/datos:/datos" \
  franki678/bigdata-sandbox:base
```

```
docker exec bigdata datanode
```

- `DATANODES`, `REPLICATION`, `BLOCK_SIZE`, `SERVICIOS` y `MONGODB` se cambian cuando quieras. La replicación y el bloque nuevos valen para lo que subas desde ese momento.
- `NAMENODES` no: queda grabado en el volumen la primera vez. Para probar alta disponibilidad se usa otro volumen (parte 14).
- Para pasar a una imagen nueva es lo mismo: `docker pull` (o el archivo del aula), `docker rm -f bigdata` y el mismo `docker run`.

---

7. HDFS

Desde acá, todo va **adentro** del contenedor, parado en `/datos`, salvo que el bloque diga otra cosa.

- El **NameNode** guarda los metadatos: qué archivos hay, en qué bloques está partido cada uno y en qué DataNodes está cada bloque.
- Los **DataNodes** guardan los bloques. Cada bloque se copia en `REPLICATION` DataNodes distintos.
- HDFS no es tu carpeta: se mira con `hdfs dfs -ls`, no con `ls`. El camino de un archivo es: lo dejás en `datos/` → `hdfs dfs -put` lo sube → se procesa → `hdfs dfs -get` baja el resultado.

Un archivo chico

```
cat ejemplos/ventas.csv
hdfs dfs -mkdir -p /user/root/prueba
hdfs dfs -put -f ejemplos/ventas.csv /user/root/prueba/
hdfs dfs -ls /user/root/prueba
hdfs dfs -cat /user/root/prueba/ventas.csv
```

`-put -f` pisa el archivo si ya estaba. En `-ls`, la segunda columna es la replicación (2). Las rutas sin `/` adelante son relativas a `/user/root`.

El dataset de 200.000 ventas

`generar_ventas.py` arma un CSV de ventas de un almacén, siempre el mismo: a todos les dan los mismos números.

```
python3 ejemplos/generar_ventas.py
ls -lh ventas_grandes.csv
head -3 ventas_grandes.csv
```

Se sube a `/user/root/ventas`, la carpeta que leen los ejemplos de MapReduce, Pig, Hive y Spark:

```
hdfs dfs -mkdir -p /user/root/ventas
hdfs dfs -put -f ventas_grandes.csv /user/root/ventas/
hdfs dfs -ls -h /user/root/ventas
```

¿Llegó igual? La huella md5 de tu copia y la de HDFS tienen que coincidir:

```
md5sum ventas_grandes.csv
hdfs dfs -cat /user/root/ventas/ventas_grandes.csv | md5sum
```

Bloques y réplicas

```
hdfs fsck /user/root/ventas/ventas_grandes.csv -files -blocks -locations
```

Con bloques de 1 MB, el archivo de ~6 MB queda en 6 bloques (`blk_...`); con el bloque por defecto (128 MB) sería uno. Cada bloque dice `Live_repl=2` y en qué DataNodes está. Al final: `Status: HEALTHY`. En http://localhost:9870 → *Utilities* → *Browse the file system* → el archivo → *Block information* se ve lo mismo.

El resumen del cluster:

```
hdfs dfsadmin -report
```

Cambiar la replicación

```
hdfs dfs -setrep -w 3 /user/root/prueba/ventas.csv
hdfs dfs -ls /user/root/prueba
```

`-w` espera a que estén todas las copias. El tamaño de bloque, en cambio, no se cambia: hay que volver a subir el archivo.

Bajar y borrar

```
hdfs dfs -get -f /user/root/prueba/ventas.csv copia.csv
ls -l copia.csv
hdfs dfs -rm -r /user/root/prueba
rm copia.csv
```

Se cae un DataNode

`datanode stop N` apaga un DataNode (simula que se cae ese nodo). Como cada bloque tiene 2 copias, el archivo se sigue leyendo entero:

```
datanode stop 2
datanode
hdfs dfs -cat /user/root/ventas/ventas_grandes.csv | md5sum
```

A los ~40 s el NameNode lo da por muerto (*Datanodes* → *Dead*) y vuelve a copiar los bloques que quedaron con una sola copia (`hdfs fsck / | tail -20` muestra cómo bajan los *Under-replicated blocks*). Lo prendemos otra vez:

```
datanode start 2
datanode
```

`datanode start N` con un N mayor que `DATANODES` suma un nodo nuevo, hasta 9.

Comando	Qué hace
`hdfs dfs -ls [-h] <carpeta>`	Lista
`hdfs dfs -mkdir -p <carpeta>`	Crea una carpeta
`hdfs dfs -put [-f] <local> <hdfs>`	Sube
`hdfs dfs -get [-f] <hdfs> <local>`	Baja
`hdfs dfs -cat <archivo>`	Muestra el contenido
`hdfs dfs -rm [-r] <ruta>`	Borra
`hdfs dfs -du -h <carpeta>`	Cuánto ocupa
`hdfs dfs -setrep -w N <ruta>`	Cambia la replicación
`hdfs fsck <ruta> -files -blocks -locations`	Bloques y dónde está cada copia
`hdfs dfsadmin -report`	Estado de cada DataNode
`hdfs dfsadmin -safemode get`	Si HDFS está en safe mode
`datanode` , `datanode stop N` , `datanode start N`	Lista, apaga o prende el DataNode N

8. YARN y MapReduce

YARN reparte memoria y CPU entre los trabajos; su página (http://localhost:8088) muestra todo lo que corre. **MapReduce** procesa en tres pasos: *map* (cada mapper lee un bloque y emite pares clave → valor), *shuffle* (Hadoop agrupa y ordena por clave) y *reduce* (cada reducer resume los valores de una clave).

MapReduce sin Hadoop

`ejemplos/mapper.py` y `ejemplos/reducer.py` calculan las ventas por producto. Se prueban con una tubería, donde `sort` hace de shuffle:

```
head -4 ejemplos/ventas.csv | python3 ejemplos/mapper.py
echo ----
cat ejemplos/ventas.csv | python3 ejemplos/mapper.py | sort | python3 ejemplos/reducer.py
```

```
yerba    3        7500
mate     1        8000
yerba    5        12500
----
bombilla 2        6000
mate     3        24000
termo    1        35000
yerba    9        22500
```

Wordcount en el cluster

El ejemplo en Java que trae Hadoop: cuántas veces aparece cada palabra.

```
hdfs dfs -put -f ejemplos/texto.txt /user/root/
hdfs dfs -rm -r -f /user/root/wc-out
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar \
  wordcount /user/root/texto.txt /user/root/wc-out
hdfs dfs -ls /user/root/wc-out
hdfs dfs -cat /user/root/wc-out/part-r-00000
```

MapReduce no pisa resultados: si la carpeta de salida existe, falla con `Output directory ... already exists` (por eso el `-rm -r -f`). La salida tiene un `part-r-NNNNN` por reducer y un `_SUCCESS` vacío. Terminado, el detalle está en http://localhost:19888.

Ventas por producto con Hadoop Streaming

Streaming corre cualquier programa como mapper y reducer: los mismos dos scripts, ahora sobre las 200.000 ventas, repartidos en el cluster.

```
hdfs dfs -rm -r -f /user/root/resultados/mr_por_producto
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
  -files ejemplos/mapper.py,ejemplos/reducer.py \
  -mapper "python3 mapper.py" -reducer "python3 reducer.py" \
  -input /user/root/ventas -output /user/root/resultados/mr_por_producto
hdfs dfs -cat /user/root/resultados/mr_por_producto/part-*
```

Resultado (producto, unidades, facturado; el orden puede variar):

```
azucar   89796    107755200
bombilla 36281    108843000
cafe     29471    191561500
facturas 121087   108978300
galletitas        82979    149362200
mate     48154    385232000
termo    11793    412755000
yerba    179512   448780000
```

En el log aparece `number of splits:6` : 6 mappers, uno por bloque de 1 MB. Estos números tienen que salir igual con Pig, Hive y Spark: es el mismo cálculo con cuatro herramientas.

YARN desde la terminal

```
yarn node -list
yarn application -list -appStates FINISHED
```

Comando	Qué hace
`yarn node -list`	Los NodeManagers
`yarn application -list`	Los trabajos corriendo
`yarn application -kill <id>`	Corta un trabajo (`application_...`)
`yarn logs -applicationId <id>`	Los logs de un trabajo terminado
`mapred job -list`	Los trabajos MapReduce corriendo

---

9. Pig

Pig Latin escribe el procesamiento paso a paso (cargar, filtrar, agrupar, sumar) y Pig lo traduce a trabajos MapReduce. Cada línea crea una relación; nada corre hasta un `DUMP` (mostrar) o un `STORE` (guardar). Pig no es un servicio: es un comando, y con `pig -x local` corre sin cluster.

```
cat ejemplos/wordcount.pig
hdfs dfs -put -f ejemplos/texto.txt /user/root/
pig ejemplos/wordcount.pig
```

Sin cluster, sobre el archivo de tu carpeta (no usa HDFS ni YARN):

```
pig -x local -param entrada=ejemplos/texto.txt ejemplos/wordcount.pig
```

Las ventas por producto, guardadas en `/user/root/resultados/pig_por_producto`:

```
pig ejemplos/ventas.pig
hdfs dfs -cat /user/root/resultados/pig_por_producto/part-*
```

Tu propio script

Guardá esto como `datos/centro.pig` y corrélo con `pig -x local centro.pig`:

```
ventas = LOAD 'ejemplos/ventas.csv' USING PigStorage(',')
         AS (fecha:chararray, sucursal:chararray, producto:chararray,
             cantidad:int, precio:int);
datos  = FILTER ventas BY fecha != 'fecha';
centro = FILTER datos BY sucursal == 'Centro';
con_importe = FOREACH centro GENERATE fecha, producto, cantidad, cantidad * precio AS
importe;
DUMP con_importe;
```

```
(2026-10-01,yerba,3,7500)
(2026-10-02,yerba,5,12500)
(2026-10-03,termo,1,35000)
```

Para el cluster: `LOAD '/user/root/ventas'` y `pig centro.pig`. La consola interactiva es `pig -x local` (o `pig`); se sale con `quit`.

Instrucción	Qué hace	En SQL
`LOAD 'ruta' USING PigStorage(',') AS (...)`	Lee con su esquema	`FROM`
`FILTER r BY cond`	Filtra	`WHERE`
`FOREACH r GENERATE a, b*c AS x`	Elige o calcula columnas	`SELECT`
`GROUP r BY col`	Agrupa	`GROUP BY`
`SUM`, `COUNT`, `AVG`	Resumen de cada grupo	`SUM`, `COUNT`…
`ORDER r BY col DESC`	Ordena	`ORDER BY`
`JOIN a BY x, b BY y`	Une	`JOIN`
`DUMP r` / `STORE r INTO 'ruta'`	Muestra / guarda (ejecuta)	

---

10. Hive

SQL sobre los archivos de HDFS. El **Metastore** (puerto 9083) guarda el esquema de cada tabla; **HiveServer2** (puerto 10000, web en 10002) atiende a los clientes; **beeline** es el cliente de consola; cada consulta corre como un trabajo **Tez** en YARN. Necesita `SERVICIOS=todo` (el valor por defecto).

```
beeline -u jdbc:hive2://localhost:10000 -f ejemplos/ventas.hql
```

`ventas.hql` crea la tabla **externa** `ventas` sobre `/user/root/ventas` (Hive sólo anota el esquema: un `DROP TABLE` no borra los archivos) y calcula por producto y por sucursal.

La consola:

```
beeline -u jdbc:hive2://localhost:10000
```

Cada consulta termina con `;` y se sale con `!quit`. Probá estas (también sirven en un archivo `.sql` con `beeline -u jdbc:hive2://localhost:10000 -f archivo.sql`):

```
SHOW TABLES;
DESCRIBE ventas;
SELECT COUNT(*) FROM ventas;
SELECT * FROM ventas LIMIT 5;
SELECT sucursal, COUNT(*) AS tickets, SUM(CAST(cantidad AS BIGINT) * precio) AS facturado
FROM ventas
GROUP BY sucursal
ORDER BY facturado DESC;
```

`CAST(... AS BIGINT)`: la suma de 200.000 importes no entra en un `INT`.

Una tabla **administrada**: con `CREATE TABLE ... AS SELECT` Hive guarda los archivos en su *warehouse* (`/user/hive/warehouse`) y `DROP TABLE` los borra.

```
DROP TABLE IF EXISTS ventas_por_producto;
CREATE TABLE ventas_por_producto AS
SELECT producto, SUM(cantidad) AS unidades, SUM(CAST(cantidad AS BIGINT) * precio) AS
facturado
FROM ventas
GROUP BY producto;
SELECT * FROM ventas_por_producto ORDER BY facturado DESC;
SHOW TABLES;
```

```
hdfs dfs -ls -R /user/hive/warehouse
```

Hive y Spark comparten el Metastore: estas tablas se ven también desde Spark (parte 12).

---

11. MySQL y Sqoop

Adentro corre un MySQL (MariaDB) con la base `tienda`: la misma empresa, como base relacional aparte del cluster. La carga el primer arranque.

Tabla	Qué tiene
`productos`	Los 8 productos, con categoría y precio

Tabla	Qué tiene
`sucursales`	Las 4 sucursales
`ventas`	Las mismas 200.000 ventas, con un `id`
`resumen_categoria`	Vacía: ahí devuelve Sqoop el resultado

```
SHOW TABLES;
SELECT * FROM productos;
SELECT COUNT(*) FROM ventas;
SELECT * FROM ventas LIMIT 3;
```

La consola es `mysql tienda` (salir con `exit`). Desde tu compu: `localhost:3306`, usuario `alumno`, clave `alumno`.

Sqoop copia tablas entre una base relacional y HDFS por JDBC. Cada `import` o `export` es un trabajo MapReduce de sólo mappers. Escribe muchos `WARN`: lo que importa es el final.

```
sqoop list-databases --connect jdbc:mysql://localhost --username alumno --password alumno
sqoop list-tables --connect jdbc:mysql://localhost/tienda --username alumno --password
alumno
sqoop eval --connect jdbc:mysql://localhost/tienda --username alumno --password alumno \
  --query "SELECT * FROM productos"
```

Importar una tabla chica con un mapper (`-m 1` → un archivo):

```
sqoop import --connect jdbc:mysql://localhost/tienda --username alumno --password alumno \
  --table productos \
  --target-dir /user/root/tienda/productos --delete-target-dir \
  -m 1
hdfs dfs -cat /user/root/tienda/productos/part-m-00000
```

La tabla grande con 4 mappers: Sqoop parte la clave primaria en 4 rangos.

```
sqoop import --connect jdbc:mysql://localhost/tienda --username alumno --password alumno \
  --table ventas \
  --target-dir /user/root/tienda/ventas --delete-target-dir \
  -m 4
hdfs dfs -ls /user/root/tienda/ventas
hdfs dfs -cat /user/root/tienda/ventas/part-m-00000 | head -3
```

Procesar en Hive con un JOIN (facturación por categoría):

```
beeline -u jdbc:hive2://localhost:10000 -f ejemplos/tienda.hql
hdfs dfs -cat /user/root/resultados/hive_por_categoria/*
```

Y devolverlo a MySQL. La tabla destino tiene que existir; se vacía antes para poder repetir:

```
sqoop eval --connect jdbc:mysql://localhost/tienda --username alumno --password alumno \
  --query "TRUNCATE TABLE resumen_categoria"
sqoop export --connect jdbc:mysql://localhost/tienda --username alumno --password alumno \
  --table resumen_categoria \
  --export-dir /user/root/resultados/hive_por_categoria \
  -m 1
```

```
SELECT * FROM resumen_categoria ORDER BY facturado DESC;
```

```
categoria         unidades  facturado
accesorios        96228     906830000
infusiones        208983    640341500
almacen   172775    257117400
panaderia         121087    108978300
```

Los cuatro pasos juntos: `bash ejemplos/sqoop.sh`.

Opción	Qué hace
`--table t`	La tabla
`--target-dir` / `--export-dir`	Carpeta de HDFS destino (import) / origen (export)
`--delete-target-dir`	Pisa la carpeta destino
`-m N`	Mappers en paralelo
`--split-by col`	Columna para partir (si no hay clave primaria)
`--where "cond"`	Sólo esas filas
`--query "SELECT ... WHERE \$CONDITIONS"`	Importar una consulta

---

12. Spark

Procesamiento en memoria con Python (PySpark), sobre YARN. Un programa tiene un **driver** (tu programa) y **executors** (en contenedores de YARN). Las transformaciones no corren hasta una acción (`show` , `count` , `write`). Mientras corre: http://localhost:4040; terminado: http://localhost:18080.

```
spark-submit ejemplos/ventas.py
```

`ventas.py` hace el mismo cálculo con DataFrames, lo guarda en `/user/root/resultados/spark_por_producto` y, si existe la tabla `ventas` de Hive, agrega las ventas por sucursal con SQL.

Tu propio programa

Guardalo como `datos/mi_spark.py` y corrélo con `spark-submit mi_spark.py` :

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("mi prueba").getOrCreate()

ventas = spark.read.csv("hdfs:///user/root/ventas", header=True, inferSchema=True)
print("filas:", ventas.count())

# Ticket promedio por sucursal, sólo las ventas de yerba
(ventas.filter(F.col("producto") == "yerba")
       .withColumn("importe", F.col("cantidad") * F.col("precio"))
       .groupBy("sucursal")
       .agg(F.count("*").alias("tickets"), F.round(F.avg("importe"), 0).alias("promedio"))
       .orderBy("sucursal")
       .show())

spark.stop()
```

SQL sobre las tablas de Hive

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("sql sobre hive").getOrCreate()

spark.sql("SHOW TABLES").show()
spark.sql("""
    SELECT producto, SUM(cantidad) AS unidades, SUM(CAST(cantidad AS BIGINT) * precio) AS
facturado
    FROM ventas
    GROUP BY producto
    ORDER BY facturado DESC
""").show()

spark.stop()
```

La consola interactiva es `pyspark` (la variable `spark` ya está creada; se sale con `exit()`). Mientras está abierta ocupa memoria de YARN: si un trabajo queda en `ACCEPTED`, cerrá las consolas que no uses.

```
pyspark
```

---

13. MongoDB

Base NoSQL de documentos: en vez de filas con columnas fijas, documentos con sus propios campos, listas y subdocumentos. Viene **apagada**: se prende con `-e MONGODB=si` (el `docker run` de esta guía ya lo trae). Anda con cualquier `SERVICIOS` y la base `tienda` se carga sola la primera vez.

```
estado
```

MongoDB	SQL
colección (`pedidos`)	tabla, pero sin columnas fijas
documento	fila, cada uno con sus campos
campo	columna; puede ser una lista o un subdocumento
`_id`	clave primaria
`$lookup`	`JOIN`

La base `tienda` tiene `productos` (9, cada uno con sus propios campos), `clientes` (2.000, con campos que algunos tienen y otros no) y `pedidos` (20.000, con sus ítems adentro).

La consola es `mongosh tienda` (prompt `tienda>`, se sale con `exit`); desde tu compu, `docker exec -it bigdata mongosh tienda`.

```
mongosh tienda
```

```
show collections
db.productos.countDocuments()
db.clientes.countDocuments()
db.pedidos.countDocuments()
```

Leer y filtrar

```
db.productos.findOne({ _id: "termo" })
```

```
db.productos.find({ categoria: "accesorios" }, { precio: 1 })
db.productos.find({}, { categoria: 1, precio: 1 }).sort({ precio: -1 }).limit(3)
db.productos.find({ categoria: "infusiones" }, { _id: 1 })
```

El primer `{}` es el filtro; el segundo, la proyección (qué campos mostrar).

```
db.productos.find({ precio: { $gt: 5000 } }, { precio: 1 })
db.productos.find({ _id: { $in: ["mate", "termo", "bombilla"] } }, { precio: 1 })
db.pedidos.countDocuments({ canal: "telefono", total: { $gt: 50000 } })
db.pedidos.countDocuments({ $or: [ { canal: "telefono" }, { calificacion: 1 } ] })
```

Campos que no todos tienen, subdocumentos (con punto) y listas:

```
db.clientes.countDocuments({ email: { $exists: true } })
db.clientes.countDocuments({ mayorista: { $exists: true } })
db.pedidos.countDocuments({ cupon: { $exists: true } })
```

```
db.clientes.countDocuments({ "direcciones.ciudad": "Quilmes" })
db.productos.find({ colores: "negro" }, { colores: 1 })
db.pedidos.countDocuments({ "items.producto": "termo", "items.cantidad": { $gte: 4 } })
db.pedidos.countDocuments({ items: { $elemMatch: { producto: "termo", cantidad: { $gte: 4 }
} } })
```

Las dos con `termo` dan distinto: la primera acepta un termo y *cualquier* ítem con 4 o más; `$elemMatch` pide que el mismo ítem cumpla las dos cosas.

Escribir

En una colección de práctica (el `drop` la deja vacía para repetir):

```
db.practica.drop()
db.practica.insertOne({ _id: 1, nombre: "Ana", materias: ["Big Data"] })
db.practica.insertMany([
  { _id: 2, nombre: "Bruno", materias: ["Big Data", "Redes"], email: "bruno@correo.com" },
  { _id: 3, nombre: "Carla" }
])
db.practica.updateOne({ _id: 1 }, { $set: { email: "ana@correo.com" }, $push: { materias:
"Bases de datos" } })
db.practica.updateMany({}, { $inc: { visitas: 1 } })
db.practica.deleteOne({ _id: 3 })
db.practica.find()
```

Agregar: el GROUP BY de MongoDB

```
db.pedidos.aggregate([ { $group: { _id: null, total: { $sum: "$total" } } } ])
```

```
db.pedidos.aggregate([
  { $unwind: "$items" },
  { $group: { _id: "$items.producto",
              unidades: { $sum: "$items.cantidad" },
              facturado: { $sum: { $multiply: ["$items.cantidad", "$items.precio"] } } } },
  { $sort: { facturado: -1 } }
])
```

`$unwind` abre la lista de ítems (un documento por ítem); la yerba da 32734 unidades. Con `$lookup` se trae el nombre de cada cliente:

```
db.pedidos.aggregate([
  { $group: { _id: "$cliente_id", gastado: { $sum: "$total" }, pedidos: { $sum: 1 } } },
  { $sort: { gastado: -1 } },
  { $limit: 3 },
  { $lookup: { from: "clientes", localField: "_id", foreignField: "_id", as: "cliente" } },
  { $project: { gastado: 1, pedidos: 1, nombre: { $first: "$cliente.nombre" } } }
])
```

Índices

```
db.pedidos.getIndexes().map(i => i.name)
db.pedidos.find({ cliente_id: 42 }).explain().queryPlanner.winningPlan.inputStage.stage
db.pedidos.find({ canal: "telefono" }).explain().queryPlanner.winningPlan.stage
```

Por `cliente_id` hay índice (`IXSCAN`); por `canal` no, y recorre todo (`COLLSCAN`).

Todo junto, con su SQL equivalente al lado: `ejemplos/consultas_mongo.js`.

```
mongosh --quiet tienda ejemplos/consultas_mongo.js
```

Desde tu compu, una consulta suelta (y MongoDB Compass con `mongodb://localhost:27017`):

```
docker exec bigdata mongosh --quiet tienda --eval "db.pedidos.countDocuments()"
```

Para dejar la base como vino (borra `tienda` entera, también `practica`):

```
mongosh --quiet ejemplos/tienda_mongo.js
```

---

14. ZooKeeper y alta disponibilidad

Con un NameNode, si se cae, HDFS queda sin servicio. Con `NAMENODES=2` o `3` hay uno *active* y los demás *standby*:

Pieza	Qué hace
NameNodes (2 o 3)	Uno atiende; los otros tienen los metadatos al día
JournalNodes (3)	Guardan el registro de cambios: el activo escribe, los standby leen
ZooKeeper (2181)	Lleva el "candado" que sólo puede tener un NameNode: el activo
ZKFC (uno por NameNode)	Vigila a su NameNode; si se cae, suelta el candado y otro toma el control

`NAMENODES` queda fijo al crear el volumen, así que se arma otro cluster, `bigdata-ha`, con su propio volumen y sólo HDFS. Los dos juntos no entran en la memoria de una notebook: primero se apaga el otro. Desde tu compu:

```
docker stop bigdata
docker run -d --name bigdata-ha --stop-timeout 20 \
  -e NAMENODES=2 -e DATANODES=3 -e REPLICATION=2 -e BLOCK_SIZE=1m -e SERVICIOS=hdfs \
  -p 127.0.0.1:9870-9872:9870-9872 -p 127.0.0.1:8480-8482:8480-8482 \
  -v bigdata_ha:/hadoop \
  franki678/bigdata-sandbox:base
```

En PowerShell o CMD, en una línea sin las `\`. Esperá **Cluster LISTO** (`docker logs -f bigdata-ha`):

```
docker exec bigdata-ha namenode
```

```
  NameNode 1 (nn1)  http://localhost:9870  active
  NameNode 2 (nn2)  http://localhost:9871  standby
```

Adentro de `bigdata-ha` (`docker exec -it bigdata-ha bash`):

```
hdfs haadmin -getAllServiceState
zkCli.sh -server localhost:2181 ls /hadoop-ha/sandbox
```

`ActiveStandbyElectorLock` es el candado. Un archivo para leer después:

```
hdfs dfs -put -f ejemplos/texto.txt /user/root/
hdfs dfs -ls /user/root
```

Se apaga el activo y, a los segundos, el otro toma el control y HDFS sigue andando:

```
activo=$(namenode | awk '/active/ {print $2}')
echo "El activo es el $activo: lo apago"
namenode stop $activo
sleep 20
namenode
hdfs dfs -cat /user/root/texto.txt
```

Al prenderlo vuelve como standby (uno de los dos `start` dice que ya estaba prendido):

```
namenode start 1
namenode start 2
sleep 20
namenode
```

Volver a tu cluster (borra el de HA y su volumen):

```
docker rm -f bigdata-ha
docker volume rm bigdata_ha
docker start bigdata
```

Comando (adentro)	Qué hace
`namenode`	Lista los NameNodes y cuál es el activo
`namenode stop N` / `namenode start N`	Apaga / prende el NameNode N
`hdfs haadmin -getAllServiceState`	El estado según Hadoop
`zkCli.sh -server localhost:2181`	Consola de ZooKeeper (`ls /`, `get <nodo>`, `quit`)

---

15. Problemas comunes

Casi todo se diagnostica con esto, desde tu compu:

```
docker ps -a --filter name=bigdata
docker stats --no-stream bigdata
docker exec bigdata hdfs dfsadmin -safemode get
```

```
docker logs --tail 40 bigdata
```

Para buscar un error del arranque: `docker logs bigdata | grep -E "ERROR|AVISO"`. Adentro, los logs de cada servicio están en `/opt/hadoop/logs/` (uno por proceso: `hadoop-dn2-datanode-*.log` ...), `/opt/hive/*.log`, `/opt/spark/logs/`, `/var/log/mysql/error.log` y `/var/log/mongodb/mongod.log`.

Síntoma	Causa y arreglo
`the container name "/bigdata" is already in use`	Ya existe: `docker start bigdata`. Para recrearlo, antes `docker rm -f bigdata`
`hdfs: command not found` (o `pig`, `beeline` ...)	Estás en tu compu: primero `docker exec -it bigdata bash`
`docker: command not found`	Estás adentro: primero `exit`
`Cannot connect to the Docker daemon`	Docker Desktop no está abierto
`invalid characters for a local volume name`	El `docker run` es de otra consola
`port is already allocated`	Otro programa usa el puerto: cambiá el número del medio (`-p 127.0.0.1:18088:8088`)
Se apaga solo con `[sandbox] ERROR: ...`	Un parámetro mal escrito: el mensaje dice cuál
Se apaga solo, sin `ERROR`	Poca RAM para Docker (parte 1)
`AVISO: este volumen se creo con N NameNode(s)`	`NAMENODES` no cambia sobre un volumen existente
`Name node is in safe mode`	Recién prendido: esperá el LISTO. Si sigue, faltan DataNodes
`Output directory ... already exists`	`hdfs dfs -rm -r <carpeta>` y de nuevo
Un trabajo queda en `ACCEPTED`	YARN sin memoria libre: cerrá consolas de `pyspark` o `beeline`
`pig` se queda en `Connecting to ResourceManager`	YARN apagado (`SERVICIOS=hdfs`): `pig -x local` o recreá con `mapreduce` o `todo`

Síntoma	Causa y arreglo
`beeline` dice `Connection refused`	HiveServer2 todavía arranca o `SERVICIOS` no es `todo`
`sqoop` dice `Communications link failure`	MySQL no corre (`SERVICIOS=hdfs`)
`sqoop export` con `Duplicate entry`	`TRUNCATE TABLE` antes
`mongosh` dice `ECONNREFUSED`	MongoDB apagado: recreá con `-e MONGODB=si`
`AVISO: este procesador no tiene AVX`	Sin MongoDB en esa compu; lo demás anda
`docker pull` dice `toomanyrequests`	Límite de Docker Hub sin cuenta: esperá, `docker login` o el archivo del aula
`ModuleNotFoundError: No module named 'pandas'`	pandas y scikit-learn vienen en la versión `ml`

Venís de la imagen anterior

Hasta octubre de 2026, `latest` era otra imagen (Ubuntu 20.04, sin MongoDB). Para saber cuál tenés:

```
docker exec bigdata grep VERSION= /etc/os-release
```

La nueva dice `24.04` y `estado` termina con `Version de la imagen: base`. Para pasar: tener la imagen nueva, `docker rm -f bigdata` y el `docker run` de la parte 4, con el mismo volumen (MySQL se pone al día solo) o con uno nuevo (empezar limpio).

Si el log dice `MySQL NO ARRANCO: este volumen es de la imagen anterior`, el volumen viejo se apagó de golpe. Si lo de MySQL no importa:

```
docker exec bigdata rm -rf /hadoop/mysql
docker restart bigdata
```

Para no perder nada, el mismo mensaje del log explica cómo cerrarlo bien con la imagen anterior (por su digest) antes de volver a la nueva.

---

16. Empezar de cero

Borra **todo** (HDFS, tablas de Hive, bases MySQL y MongoDB). Los archivos de tu carpeta `datos/` no se tocan. Desde tu compu:

```
docker rm -f bigdata
docker volume rm bigdata_hadoop
```

Y otra vez el `docker run` de la parte 4. Si `docker volume rm` dice `volume is in use`, el contenedor sigue existiendo: el `docker rm -f` va primero.